H&B

ОСНОВЫ AI-ВЕБ-РАЗРАБОТКИ

От устройства проекта до проверенного интернет-магазина

Учебник для студента и начинающего сотрудника

18 глав · 100 терминов · оригинальные примеры · упражнения и разборы

Редакция 14 ноября 2026
H&B Learning Department

Изучать последовательно, применять небольшими шагами. ИИ помогает реализовывать; ответственность за проверку и границы доступа остаётся у человека.

Содержание

Как читать эту книгу 5

Глава 1. Файл, папка и путь: где живёт сайт 5

От содержимого к адресу . . . 5

Почему картинка не находится . . . 6

Безопасное первое изменение . . . 6

Глава 2. Проект, зависимости, сборка и выполнение 7

Проект больше одной страницы . . . 7

Build не равен runtime . . . 7

Воспроизводимость вместо удачного совпадения . . . 8

Глава 3. Git и diff: как видеть последствия работы ИИ 9

История, а не запасные папки . . . 9

Читайте смысл, а не только цвет . . . 9

Точка восстановления . . . 9

Глава 4. Браузер, сервер, HTTP, DNS и HTTPS 11

Путь от имени к странице . . . 11

Читаем сообщения . . . 11

Диагностика по слоям . . . 11

Глава 5. HTML и DOM: смысл раньше украшения 13

Разметка как структура . . . 13

Исходный HTML и живое дерево . . . 13

Формы и имена . . . 14

Глава 6. CSS, адаптивность и композиция 15

Коробка каждого элемента . . . 15

Почему правило не сработало . . . 15

Композиция как управление вниманием . . . 15

Глава 7. JavaScript: значения, условия и функции 17

От текста к данным . . . 17

Правило выражается функцией . . . 17

Проверяйте границы и преобразования . . . 17

Глава 8. Объекты, массивы и модель каталога 19

Данные должны быть объяснимыми . . . 19

Идентификатор не равен номеру на экране . . . 19

Преобразование и производные значения . . . 19

Глава 9. Асинхронность: загрузка не происходит мгновенно 21

Обещание будущего результата . . . 21

Состояния и гонки . . . 21

Ошибка — часть продукта . . . 22

Глава 10. TypeScript: договор о форме данных 23

Зачем нужны типы, если приложение уже работает . . . 23

Как читать ошибку типов . . . 23

Глава 11. React: компоненты, props и состояние 25

От повторяющейся разметки к повторяемому решению . . . 25

Где должно жить состояние . . . 25

Глава 12. API, валидация и секреты: кому можно доверять 27

Контракт важнее красивого ответа . . . 27

Авторизация — не скрытая кнопка . . . 27

Глава 13. База данных, SQL, ограничения и миграции 28

Когда массива уже недостаточно . . . 28

Транзакция защищает связанное действие . . . 28

Глава 14. Интернет-магазин как последовательность состояний 29

Товар, вариант и позиция — разные понятия . . . 29

Повтор запроса и повторная оплата . . . 29

Матрица сценариев магазина . . . 29

Глава 15. UI, UX, доступность и движение 31

Красота должна помогать действию . . . 31

Семантика и управление . . . 31

Авторская концепция и анимация . . . 31

Глава 16. Архитектура проекта и управление агентами 32

Архитектура — распределение ответственности . . . 32

Проверяемое ТЗ . . . 32

Агент — исполнитель роли, а не источник истины . . . 32

Глава 17. Тестирование и отладка без гадания 33

Ожидание нужно записать до запуска . . . 33

Найти причину, а не подавить сообщение . . . 33

Глава 18. Публикация, передача и рабочая дисциплина H&B 34

Готовность существует в нескольких состояниях . . . 34

Что передаёт сотрудник Герману . . . 34

Ограниченная бесплатная демонстрация . . . 34

Словарь: 100 терминов для ежедневной работы 35

Итоговая самостоятельная проверка 38

Источники и дальнейшее чтение 39

Самостоятельный учебник для начинающего разработчика интернет-магазинов

Редакция: 13 ноября 2026 года. H&B Learning Department.

Как читать эту книгу

Эта книга объясняет устройство веб-проекта, а не обещает превратить новичка в универсального старшего разработчика за восемь недель. Её можно изучать рядом с практической программой H&B, возвращаясь к нужной главе перед изменением проекта. Здесь важнее понимать небольшое решение и уметь проверить его, чем быстро получить большой объём кода от модели.

Сквозной пример — FORMA, вымышленный магазин предметов для рабочего пространства. Названия, ассортимент, идентификаторы, цены, изображения и заказы во всех примерах демонстрационные. Реальные персональные данные и деньги не используются. Код написан специально для учебника; отдельные фрагменты иллюстрируют понятие и не образуют готовую коммерческую платформу. Серверные контракты являются проектными примерами, а не работающим внешним сервисом.

Рабочая схема H&B: Герман ведёт клиента, согласует объём и принимает результат; сотрудник реализует, проверяет и передаёт доказательства. ИИ помогает искать, объяснять и менять код, но не выдаёт себе допуск к оплатам, клиентским доступам и публикации. Описанные диалоговые действия не означают, что будущая учебная оболочка уже реализована. Доступность конкретного инструмента проверяется в подготовленной среде.

Сначала прочитайте объяснение, затем предскажите результат примера, выполните упражнение и только после своей попытки откройте подсказку. Если смысл строки непонятен, попросите ИИ объяснить её на маленьком входном наборе. Не просите сразу переписать весь проект. Метки [номер] ведут к первичным публичным документам в конце книги; педагогические рекомендации и сценарии FORMA являются авторскими.

Глава 1. Файл, папка и путь: где живёт сайт

От содержимого к адресу

Файл — именованное содержимое, сохранённое в файловой системе. В текстовом файле лежат символы: заголовок, инструкция программы или настройки. Изображение тоже файл, но его внутреннее представление не похоже на обычный текст. Папка объединяет файлы и другие папки. Поэтому сайт удобно понимать не как один красивый экран, а как набор связанных материалов: разметка, стили, логика, картинки и данные.[1]

Расширение после последней точки помогает человеку и инструментам предположить назначение файла. В `card.html` ожидается разметка, в `card.css` — оформление, в `cart.js` — программа. Переименование фотографии из `photo.jpg` в `photo.html` не превращает изображение в страницу. Тип содержимого и имя связаны соглашениями, но не магией. Когда ИИ предлагает создать файл, спросите, кто будет его читать и каким способом он попадёт в приложение.

Путь описывает расположение. Абсолютный путь начинается от корня файловой системы, например `/Users/student/forma/src/cart.js`; это вымышленный учебный адрес. Относительный путь вычисляется от текущей точки: `src/cart.js`, `./cart.js`, `../images/lamp.svg`. Точка означает текущую папку, две точки — родительскую. Одинаковый относительный путь из разных папок может привести к разным файлам. Сначала называйте базовую папку, затем обсуждайте путь.[1]

Почему картинка не находится

Представьте структуру ниже. Из `index.html` изображение доступно как `images/lamp.svg`. Из файла `styles/main.css` относительная ссылка на ту же картинку обычно будет `../images/lamp.svg`, потому что отсчёт идёт от файла стилей. Ошибка в одной букве или регистре способна остаться незаметной на одной файловой системе и проявиться после публикации. Привычка использовать простые согласованные имена экономит время расследования.[1]

```
forma-demo/
  index.html
  images/
    lamp.svg
  styles/
    main.css
  src/
    cart.js
```

Не путайте путь на диске и адрес страницы в сети. `src/cart.js` — расположение исходника внутри проекта, а `/catalog` в адресной строке может быть маршрутом приложения, которому вообще не соответствует отдельная папка. Браузер получает ресурсы через сервер и не обязан видеть внутреннее устройство проекта. Это различие станет важным, когда появятся сборка и маршрутизация.

Безопасное первое изменение

До редактирования определите разрешённую рабочую папку. Попросите помощника перечислить её верхний уровень и объяснить назначение основных частей. Не давайте задачу «найди всё на компьютере». Домашний каталог может содержать чужие проекты, доступы и личные материалы. Если нужного файла нет, это повод проверить предположение, а не расширять область поиска без разрешения.

Практический приём: найдите видимый заголовок по точной фразе, откройте найденный исходник, измените только текст и проверьте страницу. Если поиск вернул несколько совпадений, выясните, какое используется при запуске. Копия в папке собранного результата может быть перезаписана следующей сборкой. Сохранение файла ещё не доказывает, что браузер показывает именно его.

Упражнение 1. Нарисуйте дерево из разметки, стилей и двух изображений. Запишите путь к картинке из HTML и из CSS. Объясните, почему одинаковые строки могут быть неправильными.

Подсказка. Пометьте папку каждого исходного файла. От неё пройдите по дереву к картинке, считая подъёмы через `..`. Проверка — фактическая загрузка изображения, а не только уверенность в нарисованном дереве.

Глава 2. Проект, зависимости, сборка и выполнение

Проект больше одной страницы

Проект — организованная система исходников, данных, инструкций и проверок, решающая ограниченную задачу. README помогает понять запуск. Каталог исходников содержит редактируемый код. Тесты фиксируют ожидаемое поведение. Конфигурация сообщает инструментам, как работать. Эти части полезно найти до первой просьбы к ИИ: иначе помощник может создать второй способ запуска рядом с уже существующим.

Зависимость — чужой пакет, который проект использует для разработки или работы. Вместо написания библиотеки интерфейса с нуля команда подключает готовую. Это экономит усилия, но добавляет необходимость понимать назначение, совместимость и обновления. В проектах npm описание пакета и команды обычно находятся в `package.json`; это именно JSON с установленными правилами записи, а не произвольный объект JavaScript.[2]

```
{
  "name": "forma-learning-demo",
  "private": true,
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  }
}
```

Это сокращённый пример раздела команд, не полный устанавливаемый шаблон: зависимости намеренно не указаны. Он показывает соответствие имени и действия. `npm run build` вызовет значение `build`, если такой проект подготовлен. В вашем шаблоне команды могут отличаться. Ученик должен сначала прочитать реальные инструкции и файл пакета, а не запускать знакомую команду наугад.[2]

Build не равен runtime

Build — подготовка исходников к распространению: инструменты могут преобразовать синтаксис, объединить части, обработать стили и назначить итоговым ресурсам имена. Runtime — среда и время, в которых программа уже выполняется. JavaScript интерфейса работает в браузере; серверный JavaScript может работать в Node.js. Наличие одного языка не делает эти среды одинаковыми: доступные возможности и доверенные данные различаются.

В типовом статическом проекте Vite сборка создаёт набор ресурсов для размещения на сервере. Однако успешная сборка не доказывает, что правильно работает корзина, читается текст или доступна картинка по опубликованному пути. Документация Vite отдельно рассматривает публичный базовый путь: размещение сайта в подпапке требует учитывать адреса ресурсов.[24]

Разделяйте три наблюдения: «процесс запустился», «страница открылась», «нужный сценарий прошёл». Первое не гарантирует второго, второе — третьего. Сервер разработки может работать, но браузер открывает другой порт. Страница может появиться, но запрос к API возвращает ошибку. Чем точнее названа стадия, тем меньше бесполезных изменений предложит модель.

Воспроизводимость вместо удачного совпадения

Файл фиксации зависимостей помогает закрепить конкретное разрешение пакетов. Не просите ИИ удалять его просто ради исчезновения ошибки. Новая установка может изменить окружение и скрыть причину. Перед обновлением полезно записать состояние проекта, цель и способ повторной проверки. Изменение инструмента сборки — отдельная задача, а не незаметная часть смены цвета кнопки.

Когда команда завершилась ошибкой, прочитайте начало проблемы и итоговый код выхода. Последняя строка иногда лишь сообщает об общем провале, а конкретный файл указан выше. Помощнику передавайте необходимый безопасный фрагмент вывода, а не полный дамп окружения. Пути и секреты в журнале нужно проверять перед отправкой.

Упражнение 2. Найдите в учебном проекте команду разработки, сборки и тестов. Если тестовой команды нет, так и напишите. Объясните, какой факт подтверждает каждая команда.

Ответ. Команда разработки даёт среду предпросмотра, сборка проверяет возможность получить выходные ресурсы, тесты проверяют только закодированные ожидания. Ни одна из них отдельно не подтверждает приёмку всего магазина.

Глава 3. Git и diff: как видеть последствия работы ИИ

История, а не запасные папки

Git хранит историю состояний проекта и помогает сравнивать изменения. Коммит — зафиксированный снимок с сообщением о смысле работы. Ветка обозначает линию разработки. Для новичка важнее не сложная стратегия веток, а возможность точно ответить: от какой рабочей версии началось изменение, какие файлы затронуты и как вернуться к известному состоянию без потери чужой работы.

Diff показывает различия между выбранными состояниями. Обычный `git diff` сравнивает рабочие правки с индексом; подготовленные к коммиту изменения смотрят через `git diff --cached`. Поэтому пустой вывод одной команды не всегда означает отсутствие работы: изменения могли быть подготовлены отдельно. Сначала смотрят статус, затем выбирают правильное сравнение.[25]

```
git status --short
git diff --stat
git diff -- src/cart.js
git diff --cached
```

Эти команды предназначены для чтения. Они не дают разрешения делать коммит или отправлять изменения. Выполняйте их через доступный инструмент оболочки в разрешённом проекте. Если история уже содержит чужие незавершённые правки, помощник должен их отделить от своих и не «приводить всё к порядку» без согласования.

Читайте смысл, а не только цвет

В стандартном текстовом diff строка с минусом удалена, с плюсом добавлена; строки рядом дают контекст. Изменение условия с `quantity > 0` на `quantity >= 0` выглядит маленьким, но меняет допустимое поведение. Большой новый компонент может быть безобидным отображением, а одна строка отключения проверки — серьёзным риском. Оценивать работу по числу строк нельзя.

Чтение удобно вести в три прохода. Сначала перечень файлов: ожидаемы ли они для задачи? Затем ответственность: не переехали ли расчёты или секреты в браузер? Наконец детали: как изменились условия, сообщения, типы и тесты? Если помощник поменял настройки сборки при исправлении текста, попросите объяснить причинную необходимость. Необоснованную часть лучше отделить до продолжения.

Особое внимание удалённым тестам, пропущенным проверкам и слишком широким исключениям ошибок. Код `catch { return []; }` может превратить отказ сервера в ложное сообщение «товаров нет». Пользователь увидит аккуратную страницу, но диагностика исчезнет. Хорошее исправление сохраняет различие между пустым результатом и невозможностью получить данные.

Точка восстановления

Не используйте разрушительный откат как универсальное лекарство. Команда, которая сбрасывает всё рабочее дерево, может уничтожить несохранённую работу. Сначала выясняют, какие изменения принадлежат текущей задаче, сохраняют нужное и выбирают согласованный способ восстановления. Ученик не должен выполнять предложенную моделью опасную команду только потому, что рядом написано «безопасно».

После любой правки снова проверяйте именно новую версию. Вывод тестов до изменения не подтверждает состояние после него. В отчёте связывайте результат с версией или конкретным набором файлов. Такая дисциплина позволяет Герману принимать работу по свидетельствам, а не по тону уверенного рассказа.

Упражнение 3. ИИ исправил кнопку и одновременно удалил тест отрицательного количества. Какие вопросы задать до принятия?

Ответ. Почему тест удалён, какое требование теперь защищает границу, есть ли воспроизведение исходного дефекта? Потребуйте сохранить проверку поведения либо обоснованно заменить её более точной. Исчезновение красного теста из набора не означает исправления.

Глава 4. Браузер, сервер, HTTP, DNS и HTTPS

Путь от имени к странице

Браузер — клиентская программа, которая получает ресурсы и строит интерфейс. Сервер — программа, отвечающая на запросы; она может работать на отдельном компьютере или на вашем же устройстве в учебном режиме. Слово «сервер» обозначает и роль программы, и машину в бытовой речи, поэтому при разборе проблемы полезно уточнять, о чём говорится.[3]

Человеку удобно вводить доменное имя, а сетевое соединение устанавливается с использованием адресов. DNS помогает разрешить имя в необходимые записи, в том числе адреса. Это не папка с сайтом и не сама страница. Интернет предоставляет инфраструктуру соединения сетей; веб — один из сервисов поверх неё. Наличие доступа в интернет не гарантирует исправность конкретного домена, сервера или приложения.[4]

В упрощённой картине браузер узнаёт адрес, устанавливает защищённое соединение для HTTPS, отправляет HTTP-запрос и получает ответ. Затем загружает связанные стили, изображения и скрипты. Реальный путь включает кэши, посредников и повторное использование соединений, поэтому не следует представлять каждое открытие страницы как совершенно новый одинаковый набор операций.[3]

Читаем сообщения

Ниже учебное представление обмена, а не точная запись всех сетевых байтов. Метод говорит о действии, путь — о ресурсе, заголовки несут метаданные, тело содержит данные при необходимости. Ответ включает статус, заголовки и возможное тело. JSON — распространённый текстовый формат данных, но HTTP умеет передавать не только его.[3]

```
GET /api/products?category=light HTTP/1.1
Host: forma.example
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"items": [], "demo": true}
```

Успешный ответ с пустым списком не равен отсутствующему ресурсу. Код `404` сообщает иную ситуацию, чем `200` с `items: []`; ошибка сервера — ещё один случай. Интерфейс должен переводить техническое различие в полезное действие: сбросить фильтр, вернуться в каталог или повторить загрузку. Один вечный индикатор для всех случаев прячет проблему.

HTTPS защищает обмен в пути с помощью TLS и проверки подлинности стороны соединения, но не делает содержимое магазина честным и не исправляет ошибки бизнес-логики. Значок защищённого соединения не подтверждает правильность цены, наличие товара и качество авторизации. Не передавайте секретные данные сомнительному сервису только из-за наличия HTTPS.[3]

Диагностика по слоям

Если страница не открывается, задавайте вопросы от внешнего к внутреннему: правильно ли имя, доступен ли адрес, устанавливается ли соединение, какой HTTP-статус приходит, корректно ли приложение обрабатывает ответ? Во вкладке Network браузера смотрят адрес, метод, статус и безопасную часть данных. В Console ищут ошибки выполнения. Видимая пустота сама по себе не называет причину.

В локальном проекте проверьте, не перепутан ли порт и не остановлен ли процесс предпросмотра. Не лечите ошибку API случайным изменением DNS или отключением защиты браузера. Обход запрета может скрыть неверную архитектуру. Помощнику полезнее дать конкретный неудачный запрос и ожидаемый контракт, чем общую фразу «всё не работает».

Упражнение 4. Каталог показывает пустоту после ответа `500`. Какое различие потерял интерфейс?

Подсказка. Список мог не загрузиться вообще. Нужно состояние ошибки с понятным повтором, а не утверждение об отсутствии товаров. Сначала исправьте интерпретацию ответа, затем расследуйте серверную причину.

Глава 5. HTML и DOM: смысл раньше украшения

Разметка как структура

HTML описывает структуру и смысл содержимого: заголовки, абзацы, списки, ссылки, формы. Элемент обычно имеет открывающий и закрывающий тег, атрибуты уточняют его свойства. Некоторые элементы, например изображение, записываются без отдельного закрывающего тега. Вложенность важна: она выражает, какая часть относится к другой, а не просто создаёт отступы.[5]

Ниже полноценный маленький учебный документ. Его можно сохранить как HTML внутри разрешённого проекта. Кнопка пока ничего не добавляет — отсутствие поведения обозначено явно, а не спрятано за видом готового магазина. Изображение не используется, поэтому пример не зависит от чужого ассета.

```
<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>FORMA — учебная карточка</title>
</head>
<body>
  <main>
    <h1>Учебный каталог FORMA</h1>
    <article aria-labelledby="lamp-title">
      <h2 id="lamp-title">Лампа «Точка»</h2>
      <p>Вымышленный товар для упражнений.</p>
      <p>Демонстрационная цена: 1 290 руб.</p>
      <button type="button">Добавить в корзину</button>
    </article>
  </main>
</body>
</html>
```

Ссылка нужна для перехода, кнопка — для действия. Если весь интерфейс построен из безымянных блоков с обработчиками клика, разработчику придётся вручную восстанавливать множество полезных свойств управления. Семантические области и логичные заголовки помогают ориентироваться не только визуально, но и при использовании вспомогательных технологий.[20]

Исходный HTML и живое дерево

DOM — представление документа в памяти как дерева узлов, доступного программам. JavaScript может найти элемент, изменить текст, добавить узел или подписаться на событие. Поэтому исходный файл и текущая структура страницы не обязаны совпадать: после нажатий и запросов дерево меняется. Инструмент Elements показывает живую структуру, а не только первоначальную строку HTML.[8]

Для первого поведения выберите кнопку и подключите обработчик. Не вставляйте произвольную строку внешнего происхождения как HTML, когда нужен обычный текст. В учебном примере достаточно безопасного изменения текстового содержимого. Это показывает путь «событие — функция — изменение интерфейса», который позднее сохранится и в компонентном подходе.

```
const button = document.querySelector('button');
button.addEventListener('click', () => {
  button.textContent = 'Учебное добавление выполнено';
});
```

Фрагмент предполагает, что кнопка уже существует к моменту исполнения. Если скрипт запустить раньше построения нужной части документа, поиск может не найти элемент. Не прячьте такую проблему случайной задержкой. Выясните порядок подключения и предусмотрите подходящий момент запуска. Наличие ошибки обращения к отсутствующему элементу — наблюдение о жизненном цикле, а не доказательство неисправности браузера.

Формы и имена

Полю ввода нужна связанная подпись; placeholder не заменяет её, потому что исчезает при вводе. Связь `label` и `id` помогает понять назначение поля. Ошибка должна объяснять, что исправить, а не только окрашивать рамку. Для учебного заказа используйте вымышленные значения, не просите ученика вводить реальный телефон ради демонстрации.[21]

Упражнение 5. Замените в карточке кликабельный декоративный блок настоящей кнопкой. Проверьте клавиатуру и объясните разницу между ссылкой на товар и добавлением товара.

Ответ. Переход к отдельному адресу выражается ссылкой, изменение корзины — действием кнопки. Проверка включает фокус, активацию и понятное имя; красивый прямоугольник сам по себе не подтверждает эти свойства.

Глава 6. CSS, адаптивность и композиция

Коробка каждого элемента

CSS задаёт отображение элементов через правила: селектор выбирает цель, свойства описывают её вид. Вокруг содержимого есть внутренний отступ, граница и внешнее пространство. При обычной модели ширина содержимого и итоговый занимаемый размер могут отличаться; `box-sizing: border-box` включает внутренние отступы и границу в заданный размер. Понимание этой модели полезнее десятков случайных исправлений ширины.[6]

```
* { box-sizing: border-box; }
body {
  margin: 0;
  color: #152b34;
  background: #f5f3ee;
  font-family: system-ui, sans-serif;
  line-height: 1.55;
}
.catalog {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1.25rem;
  max-width: 72rem;
  margin-inline: auto;
  padding: 1rem;
}
.card { padding: 1.25rem; background: white; }
@media (min-width: 48rem) {
  .catalog { grid-template-columns: repeat(3, 1fr); }
}
```

Здесь одна колонка является исходным вариантом, а более широкое пространство позволяет показать три. Это не универсальная сетка для любого каталога, а понятный учебный выбор. Адаптивный интерфейс перестраивает представление под доступное место; гибкая сетка, изображения и медиазапросы помогают избежать отдельной неподвижной картинки для каждого устройства.[7]

Почему правило не сработало

На один элемент могут влиять разные правила. Смотрите в браузере вычисленное значение и источник победившего свойства. Возможно, селектор не совпал, другое правило оказалось приоритетнее, файл не загрузился или свойство применено не к тому элементу. Просьба «добавь везде important» обычно заменяет расследование и увеличивает будущие конфликты. Сначала найдите конкретное противоречие.

Flexbox удобно рассматривать для расположения элементов вдоль направления, Grid — для сеток строк и колонок. Но выбирать инструмент нужно по связи элементов, а не по моде. Кнопка с ценой внутри карточки и каталог из карточек решают разные задачи. Не создавайте сложную вложенную сетку там, где достаточно обычного потока и пары отступов.

Композиция как управление вниманием

В H&B визуальное решение начинается с вопроса покупателя. Он хочет понять предмет, его стоимость, доступность и следующее действие. Главный акцент не должен одновременно принадлежать логотипу, декоративной анимации, скидке и кнопке. Сгруппируйте связанное: название рядом с характеристикой, цена рядом с выбранным вариантом, ошибка рядом с проблемным полем. Это авторский рабочий принцип, проверяемый на сценарии.

Типографическая шкала задаёт отношения между заголовком, обычным текстом и пояснением. Сначала добейтесь читабельности и разумной длины строки, затем подбирайте выразительный характер. Русское название часто длиннее короткой английской подписи; проверяйте настоящие длинные строки, а не только удобные заглушки. Не обрезайте важные характеристики ради одинаковой высоты без понятного способа прочитать их полностью.

Проверка адаптива — не три красивых скриншота. Меняйте ширину постепенно, увеличивайте текст, открывайте длинное название и пустое состояние. Убедитесь, что нет горизонтальной прокрутки всей страницы и что основное действие не ушло за недоступную область. Значения ширин из упражнения — контрольные примеры, а не доказательство работы на любом экране.

Упражнение 6. Сделайте две композиции одной карточки без изменения данных. Попросите другого человека назвать цену и основное действие, не объясняя расположение заранее.

Подсказка. Сравнивайте не личное предпочтение, а время поиска и число ошибочных трактовок. Зафиксируйте, какое решение помогает задаче и что пришлось пожертвовать ради него.

Глава 7. JavaScript: значения, условия и функции

От текста к данным

JavaScript описывает поведение программы. Строка хранит текст, число участвует в расчёте, логическое значение представляет да или нет. Объект объединяет свойства, массив — упорядоченные элементы. `null` удобно использовать как намеренное отсутствие значения, а `undefined` часто указывает, что значение не задано. Эти различия важны при чтении ответа API и обработке пустых состояний.[9]

Переменная — имя, через которое программа обращается к значению. `const` запрещает повторное присваивание этому имени, но не превращает содержимое объекта в неизменяемое. `let` допускает новое присваивание. Хорошее имя говорит о смысле: `quantity`, а не `x`; `priceMinor`, а не неоднозначное `priceText`. Помощник должен объяснять единицы измерения так же ясно, как тип.[9]

Правило выражается функцией

Функция получает аргументы, выполняет действия и может вернуть результат. Параметры описаны при объявлении, аргументы передаются при вызове. Чистую вычислительную функцию удобно проверять отдельно от интерфейса: одинаковый вход должен давать одинаковый результат без запроса к сети и изменения страницы. Это упрощает поиск причины неверной суммы.[10]

```
function lineTotal(priceMinor, quantity) {
  if (!Number.isSafeInteger(priceMinor) || priceMinor < 0) {
    throw new Error('Некорректная учебная цена');
  }
  if (!Number.isSafeInteger(quantity) || quantity < 1) {
    throw new Error('Количество должно быть целым и положительным');
  }
  const total = priceMinor * quantity;
  if (!Number.isSafeInteger(total)) {
    throw new Error('Сумма вне учебного диапазона');
  }
  return total;
}
```

Для учебного магазина деньги хранятся в целых минимальных единицах, а подпись с валютой формируется отдельно. Функция не разбирает декоративную строку `1 290 руб.`. Такое разделение делает расчёт независимым от пробелов и оформления. Здесь ограничение безопасного целого явно включено в учебный контракт, а реальные денежные правила проекта согласуются отдельно.

Условие `if` выбирает ветку. Выражение после `return` становится результатом вызова, а дальнейшие строки этой функции уже не исполняются. `throw` передаёт управление механизму обработки ошибки. Не используйте ошибку как молчаливое значение ноль: неверный ввод и настоящая нулевая сумма имеют разный смысл. Интерфейс должен показать понятную причину отказа.

Проверяйте границы и преобразования

Значение из текстового поля часто приходит строкой. Строка `'2'` и число `2` — не одно и то же. Неявные преобразования способны дать неожиданный результат, особенно при сложении. Преобразование должно происходить в обозначенном месте с последующей проверкой диапазона. Не принимайте `Number(value)` за полную валидацию бизнес-правила.

При чтении кода идите по конкретному примеру. Назовите аргументы, затем каждое условие и получившийся результат. Если модель объясняет абстрактно, попросите трассировку для одного

корректного и одного некорректного входа. Это полезнее вопроса «всё ли правильно?», на который легко получить общее одобрение без проверки.

Упражнение 7. Предскажите поведение функции для количества ноль, отрицательного, дробного и строкового. Затем проверьте свои ожидания инструментом выполнения.

Ответ. Все перечисленные значения должны быть отвергнуты данным контрактом. Отдельно проверьте положительное целое. Запишите фактический результат, а не копируйте ожидаемый как будто это вывод запуска.

Глава 8. Объекты, массивы и модель каталога

Данные должны быть объяснимыми

Объект товара объединяет идентификатор, название и общие сведения. Вариант описывает конкретную продаваемую комбинацию: например, цвет лампы и тип основания. Если цена или наличие зависят от комбинации, их нельзя хранить только в общем тексте карточки. Для учебного проекта заранее определите, какие поля обязательны и какое значение имеет отсутствие каждого поля.

```
const variants = [
  { id: 'v-sand', productId: 'lamp',
    color: 'Песочный', priceMinor: 129000, stock: 4 },
  { id: 'v-ink', productId: 'lamp',
    color: 'Чернильный', priceMinor: 139000, stock: 0 }
];
const available = variants.filter(item => item.stock > 0);
const names = available.map(item => item.color);
const sorted = [...variants].sort(
  (a, b) => a.priceMinor - b.priceMinor
);
```

Массив хранит элементы в порядке и даёт способы преобразования. `filter` выбирает подходящие, `map` создаёт новый набор результатов, `sort` сортирует сам массив, поэтому в примере перед ним сделана поверхностная копия. Важно понимать не только результат метода, но и то, изменяет ли он исходную коллекцию. Иначе сортировка на одной странице может неожиданно повлиять на другую.[11]

Идентификатор не равен номеру на экране

Идентификатор обозначает сущность независимо от текущего порядка. Первый товар после сортировки не обязан быть тем же, что первый до неё. Поэтому номер позиции в массиве — плохая замена устойчивому идентификатору товара. Отдельный устойчивый идентификатор варианта позволяет отличать песочную лампу от чернильной, даже когда у них одно общее название.

Корзина хранит выбранные варианты и количества. В простом случае ключом позиции может быть `variantId`, если он уникален во всём каталоге. При добавлении того же варианта количество увеличивается; другой вариант создаёт отдельную позицию. Если у товара есть персонализация, ключ может требовать дополнительных параметров. Такое правило нужно написать до реализации, а не обнаруживать случайно после жалобы покупателя.

Преобразование и производные значения

Сумма корзины — производное значение: её получают из цен и количеств. Хранить одновременно позиции, сумму каждой позиции и общий итог без необходимости опасно: копии могут разойтись. В учебном клиенте удобнее пересчитывать итог из одного источника. На настоящем сервере подтверждённый заказ, напротив, должен сохранять согласованные цены и условия, чтобы последующее изменение каталога не переписало историю покупки.

Фильтры должны сочетаться предсказуемо. Запишите словами правило: категория подходит и цена в диапазоне и есть в наличии, если включён соответствующий переключатель. Затем подготовьте маленький набор, где каждое условие можно проверить отдельно. Большой случайный массив труднее анализировать: ученик не замечает, почему именно исчез конкретный товар.

Отличайте отсутствие совпадений от отсутствия исходных данных. При пустом результате фильтра предложите сбросить ограничения. При ошибке загрузки предложите повторить запрос. При пустом

каталоге у владельца может быть отдельный редакторский сценарий. Один массив `[]` без статуса не объясняет происхождение пустоты.

Упражнение 8. Опишите три добавления: песочный вариант, чернильный вариант, снова песочный. Как должен выглядеть набор позиций, если доступность временно не учитывается в этом упражнении?

Ответ. Две позиции: песочная с увеличенным количеством и чернильная отдельно. Затем верните проверку наличия: недоступный вариант должен быть отвергнут понятным сообщением, а не молча добавлен по старому сценарию.

Глава 9. Асинхронность: загрузка не происходит мгновенно

Обещание будущего результата

Сеть отвечает не сразу. Promise представляет будущий результат операции: она ещё выполняется, завершилась успешно или отклонена. `async` и `await` помогают записать последовательность работы, не превращая ожидание сети в мгновенную операцию. Пока запрос идёт, интерфейсу нужно выразить состояние ожидания и сохранить понятность действий.[12]

Успех соединения не равен успеху бизнес-операции. `fetch` обычно не отклоняет обещание только из-за HTTP-статуса ошибки, например `404`; статус требуется проверить. Чтение JSON также асинхронно и может завершиться ошибкой, если тело не соответствует формату. Поэтому несколько стадий одного запроса требуют отдельных осмысленных проверок.[13]

```
async function loadDemoCatalog(fetcher = fetch) {
  const response = await fetcher('/api/demo/products');
  if (!response.ok) {
    throw new Error(`Каталог недоступен: ${response.status}`);
  }
  const data = await response.json();
  if (!Array.isArray(data.items)) {
    throw new Error('Неверная форма учебного ответа');
  }
  return data.items;
}
```

Фрагмент проверяет только оболочку ответа, не каждое поле товара. Это намеренная граница примера, а не полная защита API. Отдельная функция принимает `fetcher`, чтобы тест мог подставить контролируемый учебный ответ без настоящей сети. Такой симулятор нужно прямо обозначать; его успех не подтверждает существование реального сервера.

Состояния и гонки

Продумайте состояния до оформления: первоначальное, загрузка, успех с данными, успех без данных, ошибка. Когда пользователь меняет фильтр несколько раз, старый запрос может завершиться позже нового. Если бездумно записывать каждый ответ в интерфейс, пользователь увидит данные предыдущего выбора. Это гонка результатов, а не «каприз браузера».

Исправление выбирают под задачу: отменяют устаревшую операцию, сравнивают идентификатор актуального запроса или используют согласованный инструмент управления серверными данными. Не нужно вводить сложную библиотеку только ради слова «асинхронность». Важно показать, какой ответ считается актуальным, и проверить ситуацию с искусственной задержкой предыдущего запроса.

Повторная отправка заказа — более опасный случай, чем повторное чтение каталога. Блокировка кнопки уменьшает случайные клики, но не гарантирует единственность серверной операции. Сеть может оборваться после создания заказа, а клиент повторит запрос. Поэтому настоящая интеграция нуждается в серверном правиле повторов и идемпотентности. Учебный экран должен честно отделять это требование от простой анимации кнопки.

Ошибка — часть продукта

Сообщение об ошибке должно говорить, что произошло, сохранились ли введённые значения и что можно сделать дальше. Не показывайте покупателю внутренний стек программы и не скрывайте отказ за ложным успехом. Для диагностики сохраняйте безопасный код события, достаточный разработчику, но не полный набор пользовательских данных.

Упражнение 9. Смоделируйте задержку, ответ `500`, неверный JSON и успешный пустой список. Запишите четыре разных ожидания до запуска.

Подсказка. Индикатор ожидания должен завершаться и при отказе. Пустой список остаётся успешным результатом, а повреждённый формат — ошибкой контракта. Если проверялся только локальный симулятор, именно это должно быть написано в отчёте.

Глава 10. TypeScript: договор о форме данных

Зачем нужны типы, если приложение уже работает

В JavaScript ошибка может проявиться только на определённом входе. Карточка отображает число, пока сервер не прислал пустое значение; расчёт выглядит правильным, пока количество не оказалось строкой. TypeScript позволяет описать ожидаемые значения и обнаруживать часть несоответствий до выполнения. Он не заменяет JavaScript: программа всё равно запускается в конкретной среде, а не внутри обещания разработчика о правильности данных.[14]

Рассмотрим учебный вариант товара. Нам нужны устойчивый идентификатор, целая цена в копейках, количество на складе и доступные изображения. Не нужно описывать двадцать полей «на будущее». Чем меньше договор, тем легче проверить его смысл.

```
type Variant = {
  id: string;
  productId: string;
  priceMinor: number;
  stock: number;
  imageUrl?: string;
};

function canAdd(variant: Variant, quantity: number): boolean {
  return Number.isInteger(quantity)
    && quantity > 0
    && quantity <= variant.stock;
}
```

Знак вопроса означает, что поле может отсутствовать. Интерфейс обязан учитывать этот случай: показать согласованный запасной визуал, а не сломанное изображение. Тип `number` сам по себе не устанавливает положительность, целочисленность или валюту. Эти ограничения в примере проверяются логикой. Различайте «значение является числом» и «значение допустимо для покупки». Это разные уровни договора.

Как читать ошибку типов

Сначала найдите место, где значение создаётся, затем место, где оно используется. Если функция ожидает `number`, а поле формы отдаёт `string`, не просите модель поставить `any`. Нужно определить границу преобразования, преобразовать ввод и проверить результат. Использование `as Variant` также не доказывает, что пришедший объект содержит необходимые поля: это заявление автора кода, не проверка ответа сети.[14]

Полезный порядок чтения сообщения: ожидаемая форма → фактическая форма → источник значения → минимальное исправление. Не меняйте все связанные типы сразу. Иногда ошибка показывает реальное противоречие требований: значение может отсутствовать, а дизайн не предусматривает такого состояния. В этом случае исправление включает продуктовый вопрос, а не только синтаксис.

Слишком сложные обобщённые типы на первом проекте мешают обучению. Начинающему достаточно понимать объект, массив, необязательное поле, объединение вариантов и возвращаемое значение функции. Например, статус запроса удобно описать как `'idle' | 'loading' | 'success' | 'error'`, а не произвольную строку. Такой договор помогает не перепутать название состояния между файлами.

Практика 10. Опишите тип позиции корзины. Уберите из него отображаемую строку цены; храните идентификатор варианта и количество. Подготовьте пример некорректного вызова, который должен обнаружить TypeScript, и пример, который остаётся типово допустимым, но нарушает бизнес-правило.

Разбор ответа. Строковое количество должно противоречить объявленному числовому типу. Отрицательное число может пройти проверку типа, поэтому функция должна отдельно отвергнуть его. В отчёте нужны два разных результата, а не общее утверждение «типы всё защищают».

Глава 11. React: компоненты, props и состояние

От повторяющейся разметки к повторяемому решению

Компонент объединяет часть интерфейса и её поведение. Карточки товаров отличаются данными, но разделяют структуру. Вместо восьми вручную скопированных карточек можно описать одну и передавать ей разные товары. Входные данные называют props. Они позволяют понять, что необходимо компоненту для отображения. Изменяемые значения, которые участвуют в поведении интерфейса, могут храниться в состоянии — state.[15]

```
type Product = { id: string; title: string; priceMinor: number };

function ProductCard({ product }: { product: Product }) {
  const [favorite, setFavorite] = useState(false);
  return (
    <article>
      <h2>{product.title}</h2>
      <p>{(product.priceMinor / 100).toFixed(2)} руб.</p>
      <button type="button" aria-pressed={favorite}
        onClick={() => setFavorite(value => !value)}>
        {favorite ? 'В избранном' : 'В избранное'}
      </button>
    </article>
  );
}
```

Это фрагмент для подготовленного React-проекта: `useState` необходимо импортировать из React. Пример показывает связь события, изменения состояния и нового отображения; он не сохраняет избранное на сервере. Обновление страницы может вернуть исходное состояние. Не называйте такую кнопку полноценным пользовательским сервисом без согласованного правила сохранения.

Где должно жить состояние

Если каждое окно страницы хранит свою независимую корзину, счётчик в шапке может расходиться с содержимым боковой панели. Общим участникам нужен согласованный источник состояния. Но это не означает, что всё нужно переносить в глобальное хранилище. Сначала определите, кому действительно требуется значение. Локально открытое меню карточки обычно не обязано быть известно всему приложению.

Не храните производные значения без причины. Если итог можно получить из позиций корзины, отдельная изменяемая копия итога создаёт возможность рассинхронизации. Это особенно заметно после удаления позиции: список обновился, а сумма осталась старой. Объясните помощнику источник истины и потребуйте проверку двух отображений одного состояния.

React использует устойчивые ключи, чтобы сопоставлять элементы списков. Идентификатор товара лучше отражает сущность, чем текущая позиция после сортировки. События следует задавать как функции, а не вызывать нужное действие прямо во время создания интерфейса. Хуки вызываются по правилам React, в предсказуемом порядке, а не случайно внутри условия. Начинающему важно уметь заметить нарушение, а не заучивать внутренний механизм библиотеки.[15]

Практика 11. Сделайте список из трёх товаров, добавьте избранное, измените порядок сортировки. Проверьте, что выбранным остаётся тот же товар. Затем объясните, где хранится состояние и что произойдёт при перезагрузке.

Ответ. Результат должен быть связан с идентификатором товара. Если отметка перескакивает на соседнюю карточку, проверьте ключи и способ хранения выбора. Если выбор пропал после перезагрузки, это не обязательно баг: сначала уточните требование к долговечности.

Глава 12. API, валидация и секреты: кому можно доверять

Контракт важнее красивого ответа

API описывает способ взаимодействия компонентов. Клиент запрашивает каталог, сервер возвращает данные; клиент отправляет выбранные варианты, сервер проверяет возможность заказа. Для каждого действия нужны форма запроса, форма ответа, значения статусов и поведение при ошибке. Текст «подключить API» без этих деталей не является проверяемым заданием.

Учебный запрос создания заказа может содержать идентификатор варианта и количество. Он не должен определять окончательную цену по произвольной строке из браузера. Пользователь контролирует свой клиент и может изменить отправляемые значения. Поэтому настоящая серверная часть заново получает доверенные сведения, проверяет ограничения и рассчитывает результат. Валидация на клиенте улучшает удобство, но не заменяет проверку на доверенной границе.[16]

```
{
  "items": [{ "variantId": "lamp-sand", "quantity": 2 }],
  "operationId": "учебный-идентификатор-действия"
}
```

Этот объект является проектным примером, не доказательством работающего обработчика. Перед реализацией запишите: что будет при неизвестном варианте, отрицательном количестве, пустой корзине, превышении наличия и повторе идентичного действия. Хороший контракт объясняет не только удачную покупку.

Авторизация — не скрытая кнопка

Аутентификация отвечает, кто обращается; авторизация — что ему разрешено. Спрятать кнопку публикации от ученика полезно для интерфейса, но недостаточно для защиты. Сервер должен отвергнуть запрещённый запрос независимо от того, откуда его отправили. Поле `role: owner` в пользовательском JSON не может само предоставить полномочия владельца.

Секретный API-ключ не размещают в исходниках браузера. Если приложение отправлено посетителю, находящаяся в нём строка не становится тайной от этого посетителя. Закрытые ключи хранятся на доверенной стороне, не выводятся в сообщения об ошибках, не попадают в учебные примеры и Git. Безопасный отчёт содержит категорию отказа и необходимые сведения для воспроизведения, но не полный дамп окружения.

Для прототипа допустим контролируемый симулятор ответа. Его нужно назвать симулятором в интерфейсе и документации. Проверка симулятора подтверждает поведение клиента при выбранном ответе; она не доказывает доступность внешнего сервиса, правильность его авторизации или коммерческую готовность интеграции.

Практика 12. Составьте пять недопустимых запросов и ожидаемые отказы. Попробуйте записать чужую роль в запросе учебной среды: доступ не должен измениться. Используйте только собственный разрешённый проект.

Ответ. Проверка должна выполняться на сервере. Удаление кнопки, переименование поля или уверенное объяснение ИИ не подтверждают запрет. Важен фактический отказ защищённой операции без изменения данных.

Глава 13. База данных, SQL, ограничения и миграции

Когда массива уже недостаточно

Массив в программе удобен для учебного каталога. Но после перезапуска он может исчезнуть, одновременно работающие пользователи могут расходиться в состоянии, а историю заказов требуется сохранять. База данных предоставляет организованное долговечное хранение и средства работы с ним. В реляционной модели данные описываются таблицами, строками, столбцами и связями.[17]

Ниже сокращённая схема для понимания. Это не готовая модель коммерческого магазина: здесь нет налогов, доставки, нескольких валют и управления складом.

```
CREATE TABLE variants (
  id TEXT PRIMARY KEY,
  product_id TEXT NOT NULL,
  price_minor BIGINT NOT NULL CHECK (price_minor >= 0),
  stock INTEGER NOT NULL CHECK (stock >= 0)
);

SELECT id, price_minor
FROM variants
WHERE stock > 0
ORDER BY price_minor ASC;
```

Первичный ключ различает строки. `NOT NULL` требует наличия значения, а `CHECK` выражает ограничение. Ограничение в БД дополняет проверку приложения: оно помогает не сохранить невозможные данные при другом пути записи. Внешний ключ связывает допустимые ссылки между таблицами; его конкретное место зависит от выбранной схемы.[26]

Транзакция защищает связанное действие

Представьте операцию из двух изменений: создать заказ и уменьшить остаток. Если второе действие не получилось, частичный результат может быть недопустим. Транзакция позволяет рассматривать связанные изменения как единое действие, которое подтверждается или отменяется. Она не угадывает бизнес-правило и не устраняет автоматически все проблемы конкурентного доступа; нужно правильно выбрать сами операции и ограничения.[18]

Не следует складывать весь магазин в одну таблицу только ради простоты первого запроса. Но и дробление на десятки сущностей без требований не помогает новичку. Начните с понятий: товар, вариант, черновик заказа, позиция заказа. Затем задайте вопрос, какая информация общая, а какая относится к конкретной покупке. Цена подтверждённой покупки не должна молча меняться вслед за сегодняшней ценой каталога.

Миграция — согласованное изменение схемы. Добавить обязательное поле к таблице с существующими строками сложнее, чем нарисовать поле в новом проекте. Нужно определить значения для старых записей, порядок выкладки и восстановление при ошибке. Поэтому ученик не запускает миграцию на клиентской БД по просьбе модели без отдельного технического допуска.

Практика 13. Нарисуйте четыре таблицы учебного заказа и связи. Укажите три невозможных состояния: отрицательный остаток, позиция без заказа, количество ноль. Назовите, на каком уровне каждое состояние проверяется.

Ответ. Проверки могут существовать и в API, и в БД. Их сочетание должно соответствовать правилам проекта. В учебном ответе важнее объяснить связь и ограничение, чем написать сложный универсальный SQL без понимания.

Глава 14. Интернет-магазин как последовательность состояний

Товар, вариант и позиция — разные понятия

Товар объединяет предложение: например, настольную лампу. Вариант описывает продаваемую комбинацию, например песочный цвет и конкретный размер. Позиция корзины связывает вариант с количеством и дополнительными параметрами выбора. Если объединять позиции только по названию товара, можно случайно заменить выбранный покупателем цвет.

Покупательский путь удобно проверить как цепочку. Каталог показывает доступные варианты поиска. Карточка объясняет предложение и выбор. Корзина позволяет сверить состав. Оформление собирает необходимые сведения и подтверждает условия. Экран результата сообщает, что действительно произошло. Не нужно начинать с анимации успеха, когда ещё не определено, что считается созданным заказом.

Для учебной FORMA договоримся: `draft` означает черновик, `submitted_demo` — отправленный учебный заказ. Мы намеренно не вводим статус `paid`, поскольку настоящие деньги не принимаются. Такой словарь помогает не выдавать нарисованный экран за завершённую интеграцию.

Повтор запроса и повторная оплата

Сеть может оборваться после того, как сервер совершил действие. Клиент не увидел подтверждения и предлагает повторить. Если обработчик безусловно создаёт новую запись, возникнут дубли. Идемпотентность описывает требование, чтобы согласованный повтор одного действия не создавал нежелательный дополнительный эффект. Нужно определить идентификатор операции, срок его действия и поведение при другом содержимом под тем же идентификатором.

В настоящих платежах возвращение пользователя на страницу успеха не является достаточным подтверждением оплаты. Платёжная система может присылать отдельные серверные уведомления. На примере документации Stripe видно, что обработчик webhook обязан учитывать проверку подлинности и повторную доставку событий; это иллюстрация класса задач, а не выбор Stripe для клиентов H&B.[19]

Сотрудник должен передать эти вопросы техническому ответственному, а не импровизировать финансовую логику. Учебный прототип может показать размещение формы и обработку сценариев, но реальный запуск требует подтверждённых условий оплаты, доставки, возвратов и согласованного содержания документов. Эти условия не следует сочинять по внешнему виду конкурента.

Матрица сценариев магазина

Проверяйте не только один товар и одну удачную покупку. Добавьте длинное название, недоступный вариант, пустой каталог, отменённое действие, повторное добавление, отказ оформления и обновление страницы. Для каждого сценария заранее запишите ожидаемое состояние и то, что не должно измениться. Именно сохранение соседних свойств часто отличает аккуратное исправление от случайно работающей демонстрации.

Практика 14. Сделайте таблицу из восьми сценариев. Для отказа оформления укажите, сохраняются ли корзина и введённые значения. Для повторного клика отделите запрет кнопки в браузере от серверного правила повторов.

Ответ. Блокировка кнопки полезна, но не доказывает защиту сервера. Сообщение об успехе должно соответствовать фактическому подтверждению. Любой непроверенный внешний шаг остаётся отдельным риском в передаче.

Глава 15. UI, UX, доступность и движение

Красота должна помогать действию

UI — интерфейс, с которым человек взаимодействует. UX — опыт решения задачи через этот интерфейс: понятно ли предложение, можно ли найти нужный товар, исправить ошибку и завершить действие. В учебном разборе не противопоставляйте красоту удобству. Хорошая композиция помогает выделить основное и уменьшить путаницу.

Визуальная иерархия отвечает, что человек замечает первым, вторым и третьим. Её создают размер, контраст, расположение и группировка. Если все карточки, кнопки и подписи одинаково кричат, приоритеты исчезают. Начните с задачи страницы: у каталога она отличается от презентационного первого экрана. Кинематографическая пауза может быть уместна в концепте и мешать выбору среди множества товаров.

Типографика включает не только название шрифта. Важны размер, насыщенность, длина строки, межстрочный интервал и контраст. Проверьте реальные длинные русские названия, а не только короткое английское слово. Система отступов помогает объединять связанное и разделять независимое; произвольные разные значения в каждом блоке делают решение трудно поддерживаемым.

Семантика и управление

Структура заголовков и смысловые области помогают ориентироваться не только зрячему посетителю, но и пользователям вспомогательных технологий. Форма требует понятных подписей полей и объяснения ошибки. Placeholder исчезает при вводе и не заменяет устойчивую подпись. Кнопка выполняет действие, ссылка ведёт к ресурсу; замена одного другим только ради внешнего вида затрудняет предсказуемое управление.[20][21]

Доступность проверяется поведением. Пройдите страницу клавиатурой: видно ли фокус, достижимы ли нужные действия, не застревает ли он внутри окна? Сравните смысл ошибки до и после исправления. Увеличьте масштаб, проверьте переносы. Не утверждайте полное соответствие стандарту по одному автоматическому числу: отдельный инструмент видит только часть возможных проблем.

Авторская концепция и анимация

Авторское решение можно сформулировать предложением: «предметы собираются в упорядоченное рабочее пространство, поэтому композиция строится на последовательном выравнивании». Затем проверить, действительно ли графика, ритм и движение поддерживают эту идею. Случайная комбинация модных градиентов и эффектов не становится концепцией только из-за необычного вида.

Предпочтение уменьшенного движения позволяет учитывать пользователей, для которых интенсивная анимация нежелательна. `prefers-reduced-motion` — механизм согласования такого поведения в CSS. Нужно сохранить смысл и работоспособность интерфейса при отключённом эффекте, а не просто спрятать важный контент вместе с анимацией.[22]

Практика 15. Возьмите один акцентный эффект и подготовьте статичный вариант. Пройдите карточку и форму клавиатурой, с длинным текстом и при узком экране. Отдельно объясните отличия концепции от трёх выбранных референсов.

Ответ. Проверяемое отличие — конкретная композиция или поведение, связанные с задачей. Фраза «такого нет на рынке» без исследования всего рынка не является доказательством. Собственные ассеты и разрешённое использование чужих материалов фиксируются отдельно.

Глава 16. Архитектура проекта и управление агентами

Архитектура — распределение ответственности

Для начинающего достаточно сначала разделить отображение, действия, данные и доверенные решения. Каталог показывает данные, функция считает результат фильтрации, клиент отправляет запрос, сервер применяет правила, БД хранит подтверждённое состояние. Смешение всех задач в одном файле затрудняет объяснение и проверку; чрезмерное дробление без причины создаёт другую сложность.

```
Покупатель
   ↓ действия и ввод
Браузер: страницы → компоненты → состояние
   ↓ запросы по договору API
Сервер: проверка доступа → валидация → правила заказа
   ↓ согласованные изменения
База данных: товары / варианты / заказы / позиции

Внешняя оплата и доставка — отдельные согласованные границы.
В учебном прототипе они не подключены.
```

Схема помогает задавать вопросы. Где пересчитывается цена? Кто знает окончательный статус заказа? Что будет, если сеть недоступна? Какой компонент отвечает за ошибку? Если на схеме нет ответа, это не всегда повод добавить библиотеку: иногда нужно уточнить правило.

Проверяемое ТЗ

Плохое требование: «сделай красивую корзину». Рабочее учебное требование: «при добавлении одного варианта дважды увеличивается его количество; другой вариант остаётся отдельной позицией; сумма пересчитывается; после удаления последней позиции появляется пустое состояние». К нему можно заранее подготовить проверку.

Каждому пункту назначьте идентификатор. Затем связывайте его с изменением и свидетельством: `CART-03 → файл расчёта → тест на два варианта → ручной прогон`. Это облегчает приёмку. Если требование изменилось, нужно изменить договор и проверки, а не только попросить модель «теперь иначе» в середине большого разговора.

Агент — исполнитель роли, а не источник истины

Оркестратор получает цель, ограничения и контекст. Планировщик помогает разбить её. Разработчик реализует согласованный шаг. Дизайн-ревьюер и QA проверяют разные стороны. Не нужно запускать все роли ради изменения подписи. Выбирайте специалиста по риску и давайте ему необходимые факты.

Минимальное задание содержит: цель, исходные наблюдения, разрешённые файлы, критерии результата, исключения и проверки. Просьба «сделай полностью идеально» не определяет ни границ, ни способа приёмки. После ответа агента всё равно нужны просмотр diff и реальное выполнение проверок. Если два агента согласились, а кнопка не работает, решающим остаётся наблюдение.

Практика 16. Разбейте изменение фильтра на три небольших задачи: правило, отображение, проверка. Для каждой запишите вход, выход и критерий. Попросите помощника сначала объяснить план без изменения файлов.

Ответ. Хорошее разделение позволяет принять отдельный результат и обнаружить место ошибки. Нельзя выдавать разработчику право изменять ядро H&B лишь потому, что в плане появились новые настройки.

Глава 17. Тестирование и отладка без гадания

Ожидание нужно записать до запуска

Тест сравнивает поведение с ожидаемым результатом. Если ожидание не связано с требованием, зелёная отметка мало говорит о полезности. Небольшой тест функции расчёта хорошо ловит ошибку суммы, но не доказывает, что покупатель видит правильную кнопку. Ручная проверка цвета, в свою очередь, не защищает расчёт при следующем изменении.

Разделяйте уровни: отдельная функция, взаимодействие частей, путь пользователя через браузер. Рекомендации Playwright предлагают тестировать наблюдаемое поведение, изолировать проверки и использовать устойчивые способы обращения к элементам. Это помогает не привязать тест к случайной внутренней разметке.[23]

```
// Оригинальный учебный пример для Node.js.
import assert from 'node:assert/strict';
const totalMinor = (priceMinor, quantity) => priceMinor * quantity;
assert.equal(totalMinor(129000, 2), 258000);
assert.equal(totalMinor(129000, 1), 129000);
```

Этот пример проверяет только умножение двух выбранных значений. Он намеренно не проверяет валидацию и границы. В настоящем наборе нужно отдельно выразить поведение при отрицательном, дробном и строковом количестве. Не делайте вид, что две строки подтверждают всю корзину.

Найти причину, а не подавить сообщение

Начните с воспроизведения: что ввели, где нажали, что ожидали и что получили? Затем уменьшите сценарий. Если ошибка появляется с двумя вариантами, нет необходимости начинать расследование с анимации главной. Найдите границу, на которой данные становятся неправильными, и сравните ожидаемое с фактическим.

Сформулируйте гипотезу: «позиции объединяются по товару, а не варианту». Предсказание: разные варианты одного товара попадут в одну строку. Проверьте маленьким набором. Только после подтверждения меняйте ключ позиции. После исправления повторите исходный сценарий и соседний: повторное добавление того же варианта всё ещё должно увеличивать количество.

Регрессия — нарушение ранее работавшего поведения. Чтобы заметить её, храните короткий список основных сценариев и запускайте связанные проверки после каждой логической редакции. Не исправляйте тест удалением ожидания только потому, что новая реализация его нарушила. Сначала выясните, действительно ли поменялось требование.

Практика 17. Зафиксируйте ошибку количества тестом, увидьте ожидаемый провал, исправьте минимальную причину и увидьте успешный повтор. Укажите вывод запуска и версию изменения. При отсутствии инструмента выполнения отметьте проверку как невыполненную.

Ответ. В отчёте должны различаться ожидаемый результат, реальный провал до исправления и реальный успешный результат после. Скопированный пример вывода не является выполненным тестом.

Глава 18. Публикация, передача и рабочая дисциплина H&B

Готовность существует в нескольких состояниях

«Код написан», «сборка прошла», «локально проверено», «владелец принял» и «опубликовано и проверено» — разные утверждения. В рабочем отчёте называйте точную стадию. Публичный адрес может отдавать старую сборку, даже когда локальная страница правильная. Проверка после публикации должна касаться самого опубликованного результата.[24]

Перед запуском нужен согласованный список: контент, сценарии, разрешённые интеграции, обработка ошибок, права доступа, восстановление, ответственный и ограничения. Не следует объявлять готовность по наличию красивого первого экрана. Для магазина особенно важно пройти выбранный путь от поиска товара до подтверждённого результата и отдельно назвать непроверенные внешние шаги.

Что передаёт сотрудник Герману

Пакет передачи включает краткий бриф, согласованное ТЗ, объяснение концепции, ссылку или разрешённый предпросмотр, матрицу требований и проверок, известные ограничения и инструкцию восстановления. Показать нужно и мобильный сценарий. Остаточный риск не прячут в конце длинного сообщения под словом «мелочь».

Пример честного статуса: «Карточка и учебная корзина проверены на трёх ширинах; отрицательное количество отвергается; настоящая оплата не подключена; публикация не выполнялась». Это полезнее обещания «магазин полностью готов», если последняя часть не доказана.

Рабочие запросы клиента проходят через Германа. Сотрудник различает исправление дефекта, уточнение существующего требования и новую функцию. Если добавлена новая платёжная схема, это не автоматически бесплатная мелкая правка. Нужно описать влияние, предложить варианты и передать решение владельцу, а не самостоятельно обещать сроки.

Ограниченная бесплатная демонстрация

Предлагаемый учебный формат бесплатного прототипа — одна основная концепция, главный экран и один ключевой сценарий на демонстрационных данных. Это предложение для согласования, не договор со всеми клиентами. Его цель — проверить направление, не бесплатно построить весь магазин. После одобрения нужен новый согласованный объём платной реализации.

Проекты H&B, по словам Германа, обычно находятся в диапазоне 60–500 тысяч рублей; сложность нельзя определять только ценой. Доля сотрудника 30–35% и целевой доход 60–70 тысяч рублей не означают гарантированные заказы. В учебном замере учитывают и бесплатные прототипы, и проверки, и переделки — иначе производительность выглядит лучше действительной.

Практика 18. Подготовьте передачу FORMA: пять выполненных требований, по одному свидетельству к каждому, три ограничения, один риск и следующий согласованный шаг. Защитите одно решение без чтения ответа ИИ. Затем внесите маленькую новую правку и повторите проверку.

Ответ. Выпуск подтверждается работой и пониманием. Двадцать четыре дня в календаре, большая история диалога и рекомендация модели не заменяют решение Германа об области допуска. После выпуска запрет менять ядро системы и чужие проекты сохраняется.

Словарь: 100 терминов для ежедневной работы

Термины ниже объясняют смысл в контексте курса. Определение не является разрешением выполнять действие в основной системе.

1. **Файл** — именованное содержимое на диске: текст, код, изображение или другой материал.

2. **Каталог / папка** — место группировки файлов и вложенных каталогов.

3. **Путь** — описание расположения файла относительно выбранной исходной точки или корня.

4. **Абсолютный путь** — путь от корня файловой системы; на другом компьютере он может отличаться.

5. **Относительный путь** — путь от текущей папки или другого определённого источника отсчёта.

6. **Расширение** — окончание имени файла, помогающее обозначить ожидаемый вид содержимого.

7. **Исходный код** — редактируемые инструкции, из которых создаётся или запускается программа.

8. **Проект** — ограниченная система кода, данных, конфигурации, инструкций и проверок.

9. **Репозиторий** — хранилище проекта с историей изменений, если используется система контроля версий.

10. **Git** — система контроля версий для сравнения, сохранения и совместной работы с изменениями.

11. **Коммит** — зафиксированное состояние изменений с пояснением их смысла.

12. **Ветка** — именованная линия разработки; в H&B действуют собственные правила работы с ветками.

13. **Diff** — представление отличий между двумя выбранными состояниями файлов.

14. **Индекс Git** — подготовленный набор изменений для следующего коммита.

15. **Конфликт** — ситуация, когда изменения нельзя объединить без содержательного решения.

16. **README** — инструкция о назначении, запуске и использовании конкретного проекта.

17. **Зависимость** — подключённый пакет чужого кода, необходимый проекту или его разработке.

18. **Пакет** — распространяемая единица кода и связанной информации с определённой версией.

19. **Менеджер пакетов** — инструмент установки и учёта зависимостей, например npm.

20. **Lockfile** — файл, фиксирующий разрешённые версии зависимостей для воспроизводимой установки.

21. **Конфигурация** — параметры и правила работы приложения или инструментов проекта.

22. **Build / сборка** — подготовка исходников к запуску или публикации установленным способом.

23. **Runtime** — среда и этап фактического исполнения программы.

24. **CLI** — способ управления программой через команды и текстовый вывод.

25. **Терминал** — интерфейс работы с командными программами; не равен разрешению выполнять любую команду.

26. **Браузер** — клиентское приложение, получающее и отображающее веб-ресурсы.

27. **Клиент** — сторона взаимодействия, которая обращается за услугой или данными.

28. **Сервер** — программа, отвечающая на запросы; иногда так называют и машину, где она работает.
29. **Хост** — компьютер или узел, на котором размещена программа или сетевой сервис.
30. **Домен** — удобное человеку сетевое имя, используемое для обращения к ресурсу.
31. **DNS** — система разрешения доменных имён и связанных с ними записей.
32. **IP-адрес** — сетевой адрес узла; не является паролем или доказательством доступности приложения.
33. **Порт** — номер сетевой точки обслуживания на узле, например для веб-сервера.
34. **URL** — адрес ресурса с указанием схемы, узла и других частей обращения.
35. **HTTP** — протокол обмена запросами и ответами между веб-участниками.
36. **HTTPS** — HTTP поверх защищённого соединения; не гарантия честности самого сайта.
37. **Запрос** — обращение клиента с методом, адресом, заголовками и возможным телом.
38. **Ответ** — результат обработки запроса, включающий статус, заголовки и возможное содержимое.
39. **HTTP-статус** — код категории результата запроса; требует интерпретации вместе с контрактом.
40. **JSON** — текстовый формат представления данных, а не исполняемый объект JavaScript.
41. **HTML** — язык смысловой структуры и содержимого веб-страницы.
42. **Тег / элемент** — запись и структурная единица разметки, задающая часть документа.
43. **Атрибут** — дополнительное свойство HTML-элемента, например язык или подпись.
44. **Семантика** — соответствие элемента его смысловому назначению, а не только внешнему виду.
45. **DOM** — программное представление структуры документа, доступное браузерному коду.
46. **CSS** — язык правил внешнего представления и расположения элементов.
47. **Селектор** — способ выбора элементов, к которым применяется CSS-правило.
48. **Класс** — именованная метка для обращения к группе элементов, в частности из CSS.
49. **Каскад** — механизм согласования конкурирующих правил оформления.
50. **Box model** — модель содержимого, внутренних отступов, границы и внешних отступов блока.
51. **Flexbox** — механизм CSS для расположения элементов вдоль выбранного направления.
52. **Grid** — механизм CSS для организации строк и столбцов макета.
53. **Адаптивность** — перестройка интерфейса под доступное место и условия использования.
54. **Breakpoint** — условная граница, при которой правила представления меняются.
55. **JavaScript** — язык программирования, применяемый в браузере и других средах выполнения.
56. **Переменная** — именованная связь с используемым программой значением.
57. **Тип** — описание категории или допустимой формы значения.
58. **Условие** — правило выбора между вариантами выполнения программы.
59. **Функция** — именованная или передаваемая операция с входом и возможным результатом.

60. **Аргумент** — конкретное значение, переданное функции при вызове.

61. **Объект** — набор связанных свойств, описывающих сущность или состояние.

62. **Массив** — упорядоченная коллекция элементов для хранения и преобразования списков.

63. **Идентификатор** — значение, устойчиво обозначающее сущность независимо от положения на экране.

64. **Событие** — произошедшее действие, например клик, ввод или завершение операции.

65. **Promise** — представление будущего результата асинхронной операции.

66. **async / await** — синтаксис организации работы с асинхронным результатом.

67. **Ошибка / exception** — сигнал о невозможности продолжить действие обычным выбранным путём.

68. **TypeScript** — язык на основе JavaScript с описанием типов и проверкой части соответствий до выполнения.

69. **React** — библиотека построения интерфейса из компонентов.

70. **Компонент** — выделенная часть интерфейса с понятной ответственностью и входными данными.

71. **Props** — входные данные React-компонента, переданные его вызывающей стороной.

72. **State** — изменяемое состояние, от которого зависит поведение или отображение компонента.

73. **Рендер** — получение представления интерфейса из текущих данных и состояния.

74. **API** — договор и средства взаимодействия между частями программной системы.

75. **Endpoint** — конкретная точка API для согласованного действия или ресурса.

76. **Валидация** — проверка значения по формату, ограничениям и правилам конкретной границы.

77. **Аутентификация** — установление того, кто обращается к системе.

78. **Авторизация** — проверка того, какие действия этому участнику разрешены.

79. **Токен** — значение для подтверждения доступа; секретный токен нельзя публиковать или вставлять в учебный отчёт.

80. **Песочница** — реально ограниченная среда выполнения; не просто отдельная папка или кнопка с замком.

81. **База данных** — организованное хранилище с правилами чтения, изменения и сохранения информации.

82. **SQL** — язык описания и обработки данных в реляционных системах.

83. **Первичный ключ** — поле или набор полей, однозначно определяющий строку таблицы.

84. **Внешний ключ** — ограничение согласованной ссылки между связанными записями.

85. **Транзакция** — объединение связанных изменений в подтверждаемое или отменяемое действие.

86. **Миграция** — согласованное изменение схемы данных с учётом существующих записей.

87. **Идемпотентность** — свойство согласованного повтора действия не создавать нежелательный новый эффект.

88. **Webhook** — доставка события от одной системы обработчику другой по установленному договору.

89. **UI** — пользовательский интерфейс: видимые элементы и средства взаимодействия.

90. **UX** — опыт решения пользовательской задачи через продукт и его сценарии.

91. **Дизайн-токен** — согласованное именованное значение оформления, например цвет или отступ.

92. **Доступность** — возможность пользоваться продуктом при различных способностях и способах взаимодействия.

93. **Фокус** — текущий элемент управления для клавиатурного взаимодействия, который нужно ясно обозначать.

94. **ТЗ** — согласованные требования, границы и условия проверки результата.

95. **Оркестратор** — координатор последовательности работы моделей, ролей и разрешённых инструментов.

96. **QA** — деятельность по обеспечению и проверке качества относительно требований и рисков.

97. **Регрессия** — нарушение ранее работавшего поведения после изменения.

98. **Отладка** — поиск и подтверждённое устранение причины наблюдаемого дефекта.

99. **Deployment / публикация** — размещение согласованной версии в целевой среде с последующей проверкой.

100. **Production** — рабочая среда с настоящими пользователями и ответственностью; не просто разблокированный режим интерфейса.

Итоговая самостоятельная проверка

Перед следующим проектом ответьте без чтения определения: где хранится цена, кто её подтверждает, что значит пустой каталог, как отличить строку от числа, что передаётся в props, где проверяется роль, как повторяется заказ, что доказывает тест и что должен получить Герман при передаче.

Если ответ получается только копированием пояснения ИИ, выберите маленький пример и пройдите его на конкретных данных. Цель учебника — не запомнить сто слов, а перестать принимать непонятные решения без проверки. В каждой практической задаче сохраняйте границы доступа, фактические свидетельства и честное описание того, что ещё не сделано.

Источники и дальнейшее чтение

Первичные публичные документы. Изложение, учебные сценарии и примеры подготовлены для H&B. Ссылки кликабельны; источник подтверждает соответствующие сведения, но не означает выполнение практического задания.

[1] MDN: files and paths

[2] npm: package.json

[3] MDN: HTTP overview

[4] MDN: Internet and DNS

[5] MDN: HTML basics

[6] MDN: CSS box model

[7] MDN: responsive design

[8] MDN: DOM introduction

[9] MDN: JavaScript grammar and types

[10] MDN: functions

[11] MDN: indexed collections

[12] MDN: promises

[13] MDN: Fetch API

[14] TypeScript: everyday types

[15] React: Quick Start

[16] OWASP: input validation

[17] PostgreSQL: tables

[18] PostgreSQL: transactions

[19] Stripe: webhooks

[20] W3C WAI: page structure

[21] W3C WAI: forms

[22] MDN: reduced motion

[23] Playwright: best practices

[24] Vite: production build

[25] Git: git diff

[26] PostgreSQL: constraints